

- **Data** is a collection of raw facts or values that represent information about an object, event, or person.
- **Information** is processed data that gives meaning.
- A **database** is an organized collection of related data that is stored electronically so it can be easily accessed, managed, updated, and retrieved.
- A **Database Management System (DBMS)** is software that allows users and applications to create, store, retrieve, update, delete, and manage data in a database.
- **Database Architecture** refers to how users, applications, and the database interact with one another. A database architecture generally consists of:
 - User (Presentation Layer): This is where users interact with the app.
 - Application Layer: This contains the business logic.
 - DBMS Layer: The DBMS receives SQL.
 - Database: Actual storage.
- Types of Database Architecture
 - One-Tier: Everything (User, Database, App) exists in one machine.
 - Two-Tier: Client directly talks to database. (Client to DB Server)
 - Two-Tier: Client, Application Server, Database Server
- Thin Client vs Thick Client
 - Thick Client: Most processing happens on client.
 - Thin Client: Most processing happens on server.
- Client-Server Model
- Database Server: A machine running a DBMS.
- Application Server: A server that contains business logic.
- The **Relational Model** is a way of organizing data into **relations (tables)** consisting of **rows (tuples)** and **columns (attributes)**, while defining relationships between them. Instead of storing everything together, we divide data into multiple related tables.
- Relation != Relationship, Relation = Table
- The relationships between tables are created using **keys**.
- A **Relation** is a table.

- A tuple is one row of a relation.
- An attribute is a column.
- A domain is the set of valid values an attribute can have.
- Schema is the structure of the database. It defines:
 - Tables
 - Columns
 - Data Types
 - Constraints
 - Relationships
- Instance means actual data stored currently.
- Degree = Number of columns.
- Cardinality = Number of rows.

KEYS

- A **key** is one or more attributes (columns) used to uniquely identify a tuple (row) in a relation (table).
- Super Key:
 - A **Super Key** is **any set of attributes that uniquely identifies a row**.
 - It may contain **extra unnecessary attributes**.
 - Super Key = Unique + May contain extra columns
- Candidate Key:
 - A **Candidate Key** is the **smallest possible Super Key**.
 - No unnecessary attributes.
 - Candidate Key = Minimal Super Key
- Primary Key:
 - A **Primary Key** is the Candidate Key chosen to uniquely identify every row.
 - Rules for selecting a candidate key as primary key:
 - Cannot be NULL
 - Cannot contain duplicates
 - Only one Primary Key per table
 - Must uniquely identify rows
- Alternate Key:
 - Candidate Keys that are **not selected** as Primary Key.
- Foreign Key:
 - A **Foreign Key** is an attribute in one table that refers to the Primary Key of another table.
 - It creates or establishes relationships between tables.

- It enforces a constraint on the attribute, every value in the foreign key must contain in the primary key attribute
- Foreign Key ensures every referenced value must exist in the parent table.
- Composite Key
 - A key made from **multiple columns**.
 - Neither StudentID nor CourseID alone is unique. Together StudentID + CourseID uniquely identify each row.
- Unique Key
 - Looks similar to Primary Key. Difference:
 - Primary Key:
 - Only One
 - No NULL
 - Unique Key:
 - Multiple Allowed
 - Usually allows NULL (because NULL values are not considered equal).
 - Both of them allow us to identify rows uniquely
- Natural Key
 - A key that already exists in the real world.
 - Eg: Aadhaar Number, PAN, Passport Number, etc.
- Surrogate Key
 - Artificial key generated by database.
 - Advantages:
 - Short
 - Never Changes
 - Fast Indexing

CONSTRAINTS

- **Constraints** are rules enforced by the DBMS on table columns to ensure the accuracy, consistency, and integrity of the data.

Types of Constraints:

- NOT NULL
 - A NOT NULL constraint ensures that a column **must always have a value**.
- UNIQUE
 - Ensures that **all values in a column (or combination of columns) are unique**.
- PRIMARY KEY

- Primary Key = Not Null + Unique
 - Only one PRIMARY KEY per table
- FOREIGN KEY
 - Foreign Key ensures every referenced value must exist in the parent table.
 - Referential Integrity: Child records cannot point to non-existing parent records.
- CHECK
 - Allows only values satisfying a condition.
 - `CHECK (age >= 18)`
- DEFAULT
 - Provides a default value when no value is supplied.
- AUTO_INCREMENT / SERIAL / IDENTITY
 - Used to serializing the rows and encoding each of them with an id which helps us with faster indexing

REFERENTIAL ACTIONS

- ON DELETE/UPDATE CASCADE
 - Delete Parent: delete children automatically
- ON DELETE/UPDATE RESTRICT
 - Parent cannot be deleted while children exist.
- ON DELETE/UPDATE SET NULL
 - Delete Parent: Foreign key becomes NULL.
- ON DELETE SET DEFAULT
 - Foreign key becomes its default value.
- NO ACTION
 - A
- **SQL (Structured Query Language)** is the standard language used to create, retrieve, update, delete, and manage data in relational databases.
- Categories of SQL Commands:
 - DDL
 - DML
 - DQL
 - DCL
 - TCL
- DDL (Data Definition Language)
 - Defines or changes the **structure** of the database.
 - DDL Commands:

- CREATE
 - ALTER
 - DROP
 - TRUNCATE
 - RENAME
- CREATE
 - Creates new objects (Tables, View, Index, etc).
- ALTER
 - Modifies an existing table.
 - Examples: Add, Drop, Rename, Modify column
- DROP
 - Deletes an object completely.
- TRUNCATE
 - Deletes **all rows** but keeps the table.
- RENAME
 - Rename table
- DML (Data Manipulation Language)
 - DDL changes **structure**.
 - DML changes **data**.
- Commands:
 - INSERT Adds rows.
 - UPDATE Modify existing rows.
 - DELETE Deletes rows.
- DQL (Data Query Language)
 - SELECT
- DQL (Data Query Language)
 - Used for permissions or permission management
 - Commands
 - GRANT Give permission.
 - REVOKE Remove permission.
- TCL (Transaction Control Language)
 - Used with transactions.
 - Commands
 - COMMIT Save changes permanently.
 - ROLLBACK Undo changes.
 - SAVEPOINT Create a checkpoint.

- CRUD Operations
 - Create INSERT
 - Read SELECT
 - Update UPDATE
 - Delete DELTE

AGGREGATE FUNCTIONS

- An **Aggregate Function** performs a calculation on **multiple rows** and returns a **single value**.

List of aggregate functions:

- COUNT
 - COUNT(column) Returns the **number of rows**.
 - COUNT(*) Counts only **non-NULL values**.
Counts every row.
- SUM Returns total.
- AVG Returns average.
- MIN Returns smallest value.
- MAX Returns largest value.
- STRING_AGG Combines multiple strings.
 - SELECT STRING_AGG(name, ', ') FROM employees;
- ARRAY_AGG Returns an array.
 - SELECT ARRAY_AGG(name) FROM employees;
-
- Aggregate functions ignore NULL values.
- Aggregate functions become much more powerful with GROUP BY.
- DISTINCT with Aggregate Functions
 - Without DISTINCT: all of the values are considered
 - With DISTINCT: Only unique values are considered
- HAVING vs WHERE
 - where: filtering row
 - having: filtering groups
- Execution Order in a SQL Query
 - FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY
- Aggregate functions often scan many rows.

- Indexes can help in some situations (especially for MIN, MAX, or indexed grouping), but many aggregate queries still require scanning a significant portion of the table.

JOINS

- A **JOIN** combines rows from **two or more tables** based on a related column.
- **INNER JOIN**
 - Returns **only matching rows** from both tables.
 - Only the intersection.
- **LEFT JOIN (or LEFT OUTER JOIN)**
 - Return all rows from LEFT table and matching rows from RIGHT table.
- **RIGHT JOIN (or RIGHT OUTER JOIN)**
 - Return all rows from RIGHT table and matching rows from LEFT table.
- **FULL OUTER JOIN**
 - Return everything, matching rows, unmatched left rows, unmatched right rows.
- **CROSS JOIN**
 - Every row with Every row.
 - Cartesian Product
 - $3 \text{ rows} * 4 \text{ rows} = 12 \text{ Rows}$
- **NATURAL JOIN**
 - Automatically joins columns with same name.
 - Eg: if both tables have column name "StudentID", db automatically joins based on that common column without explicitly mentioning it.
 - Problem: If another column accidentally has same name, results may change unexpectedly. Therefore, most developers prefer explicit joins using ON.

SET OPERATIONS

- Set Operations combine the **results of two or more SELECT queries** into a single result.
- Both queries must have same number of columns and corresponding columns should have compatible data types.
- Unlike joins, they combine **rows**, not columns.
- Types of SET Operations

- UNION
- UNION ALL
- INTERSECT
- EXCEPT (MINUS in Oracle)
- UNION
 - Combines both query results, removes duplicates.
- UNION ALL
 - Combines both query results, doesn't remove duplicates.
- INTERSECT
 - Returns rows present in both of the queries
- EXCEPT
 - Returns rows in first query that are NOT present in second query.

SUB QUERIES

- A **Subquery** is a SQL query written **inside another SQL query**.
- General Structure
 - SELECT ...
 - FROM ...
 - WHERE column OPERATOR (
 - SELECT ...
 -);
- Types of Subqueries:
 - Scalar
 - Row
 - Column
 - Table
 - Nested
 - Correlated
- A **Common Table Expression (CTE)** is a **temporary named result set** that exists only for the duration of a single SQL statement. It also supports recursion.
- CTE vs Temporary Table
 - CTE exists only during one SQL statement.
 - Temporary Table exists until session ends or dropped.

Window Function

- A **Window Function** performs calculations across a group of related rows **without collapsing them into a single row**.
 - Aggregate Function: One output per group
 - Window Function: One output per row
- The term **WINDOW** means: a subset of rows visible to the current row.
- Keyword used for window functions: OVER()
- PARTITION BY is used along side OVER()
- Types of Window Functions:
 - Ranking
 - ROW_NUMBER Assigns unique sequential numbers.
 - RANK Handles ties: skips ranks
 - DENSE_RANK Handles ties: doesn't skip ranks
 - NTILE(4) Splits rows into groups. Here: quartiles
 - Value
 - LAG Returns previous row
 - LEAD Opposite of LAG
 - FIRST_VALUE Returns first value in partition
 - LAST_VALUE Returns last value in partition
 - NTH_VALUE Returns n-th row
 - Aggregate
 - All aggregates can become window functions.
 - SUM
 - AVG
 - COUNT
 - MIN
 - MAX
 - Know how to calculate moving average and using OVER keyword for window functions
- Window Functions:
 - OVER()
 - PARTITION BY
 - ORDER BY
 - ROWS
 - Ranking
 - Value
 - Aggregate

SQL Functions

- A **SQL Function** is a built-in routine that accepts one or more inputs, performs an operation, and returns a value.

String Functions

- `SELECT LENGTH('Database');`
- `SELECT LOWER('ALICE');`
- `SELECT UPPER('alice');`
- `SELECT INITCAP('john doe');` Converts first letter in words to capital
- `SELECT CONCAT ('Alice', ' ', 'Johnson');`
- `SELECT 'Alice' || ' ' || 'Johnson';` String concat operation, same output
- `SELECT SUBSTRING('Database',1,4);` output: Data
- `SELECT REPLACE('banana','a','x');` output: bxnrxn
- `SELECT TRIM(' Alice ');`
- `SELECT LEFT('Database',4);` output: Data
- `SELECT RIGHT('Database',4);` output: base
- `SELECT POSITION('a' IN 'Database');` finds first location of substring
- `SELECT REPEAT('*',5);`
- `SELECT REVERSE('SQL');`

Numeric Functions

- `SELECT ROUND(12.567,2);`
- `SELECT CEIL(12.1);`
- `SELECT FLOOR(12.9);`
- `SELECT POWER(2,5);`
- `SELECT SQRT(81);`
- `SELECT MOD(17,5);`
- `SELECT RANDOM();` returns random number among 0 and 1.

Date & Time Functions

- `SELECT CURRENT_DATE;`
- `SELECT CURRENT_TIME;`
- `SELECT CURRENT_TIMESTAMP;`
- `SELECT NOW();`
- `SELECT AGE('2026-07-14','2005-10-10');`
- `SELECT EXTRACT(YEAR or MONTH or DAY FROM CURRENT_DATE);`

Conversion Functions

- CAST(): Convert one type into another.
 - SELECT CAST('123' AS INTEGER);
- SELECT TO_CHAR(CURRENT_DATE,'DD-MM-YYYY');
- SELECT TO_DATE('14-07-2026','DD-MM-YYYY');
- SELECT TO_NUMBER('1234','9999');

Conditional Functions

- SELECT COALESCE(NULL, NULL, 'Alice');
 - Returns first non-null value: "Alice" in this case
- SELECT NULLIF(10,10);
 - Returns NULL if 2 values are equal

VIEWS

- Can I save a SQL query and use it like a table?
 - That's exactly what a View is.
- A **View** is a **virtual table** created from the result of a SQL query.
- A VIEW doesn't store a table, instead it stores the SQL query
- Suppose we have 2 tables and we need to run the following query on a daily basis:
 - SELECT e.name, d.department, e.salary FROM employees e JOIN departments d ON e.departmentid=d.departmentid;
- Instead of running that every day, we can create a view:
 - CREATE VIEW employee_details AS SELECT e.name, d.department, e.salary FROM employees e JOIN departments d ON e.departmentid=d.departmentid;
- Now, instead of writing JOIN everytime we need that data we can simply use the following command:
 - SELECT * FROM employee_details
- Some views are updateable, when is a VIEW updateable ?
 - Uses a single base table
 - Does not use aggregate functions
 - Does not use GROUP BY, DISTINCT, UNION, complex JOINS, set operation
- DROP VIEW employee_details

- CREATE OR REPLACE VIEW employee_details AS SELECT ...
- Normal View stores query only
- Materialized View stores actual result
- REFRESH MATERIALIZED VIEW sales_summary

INDEXING

- An **Index** is a special data structure that allows the database to locate rows quickly without scanning the entire table.
- Indexes do not change our data, they only provide a faster way to find it.
- Why Not Always Use Full Table Scan?
 - Suppose there are 100 million rows, searching last row without index would need 100 million comparisons... with index 20-30 comparisons are enough

Types of Indexes

- Primary Index
 - Created on primary key automatically by db
- Secondary Index
 - Create on non-primary columns by user
 - CREATE INDEX idx_name ON employees(name);
 - Now, searching by name becomes faster
- Clustered Index
 - The table itself is stored in the index order
 - This index actually determines how rows are physically arranged
 - Only one clustering order is possible'
 - Example, students table stored by StudentID
- Non-Clustered Index
 - This creates a sperate structure.
 - Table remains unchanged, index stores pointers to rows
 - We can have many non-clustered indexes, providing fast lookups
- Composite Index
 - Indexing on multiple columns at a same time, useful when queries filter by both
 - It follows Left-Most-Prefix rule:
 - Index of A, B, C:
 - Will work efficeitn for querying on (A), (A, B) and (A, B, C) but not efficient on (B), (B, C), etc.

- Unique Index
 - Ensures no duplicates, if there's a duplicate it rejects
- Partial Index
 - Index only on selected rows
 - Smaller index, faster querying, less storage.
- Function-Based Index
 - Index based on expression
- Hash Index
 - Uses hash table, supports equality operations only
- B-Tree Index
 - Self-Balancing tree data structure, with multiple children and keys
 - It takes a bit of time for insertions, making the queries efficient
 - Supports both equality
- Creating Index:
 - `CREATE INDEX idx_salary ON employees(salary);`
- Dropping Index:
 - `DROP INDEX idx_salary;`
- Without indexes, inserting is simple... we will be directly inserting the data into table
- With indexes, we need to insert row, update index1, update index2, update index3, etc... more work = more time
- Avoid indexing for small tables
- Commands:
 - `CREATE INDEX`
 - `DROP INDEX`
 - `EXPLAIN`
 - `EXPLAIN ANALYZE`

NORMALIZATION

- **Normalization** is the process of organizing data into well-structured tables to:
 - Reduce redundancy (duplicate data)
 - Eliminate anomalies
 - Improve data integrity
 - Make the database easier to maintain
- Redundancy means storing the same information multiple times.
- Normalization prevents three anomalies:
 - Update Anomaly

- Insertion Anomaly
 - Deletion Anomaly
- Solution: Split large tables into smaller related tables
- Functional Dependency (FD)
 - An attribute **Y** is functionally dependent on **X** if each value of **X** determines exactly one value of **Y**.
- Suppose Composite Key (StudentID, CourseID) determines Grade. Here we need both columns to determine, so it is called Full Functional Dependency
- To determine StudentName we only need StudentID, so it is called partial functional dependency
- StudentID -> DepartmentID -> DepartmentName
 - StudentID and DepartmentName has indirect functional dependency

Normal Forms

- 1NF
 - Every column must contain atomic (indivisible) values, no multiple values
 - One cell should only contain a single value
- 2NF
 - 1NF + No Partial Dependency
 - Solution: Split Tables
- 3NF
 - 2NF + No Transitive Dependency
- Boyce-Codd Normal Form (BCNF)
 - For every dependency $X \Rightarrow Y$
 - X must be a super key
- 4NF
 - Multivalued dependency
- 5NF
 - Handles Join Dependency

Decomposition

- The process of breaking a large table into smaller table to achieve good database design through normalization.
- Lossless Decomposition:
 - After splitting, joining tables must recover original data
 - No information should be lost.
- Dependency Preservation:
 - All functional dependencies should remain enforceable after decomposition

Denormalization

- Merging table for performance is said to be denormalization
- Instead of JOINing 5 tables, store the data in a single table
- Faster, but more redundancy.
- Normalization is good for OLTP (Online Transaction Processing)
 - It indeed helps in optimizing the operations on large database
 - Banking, UPI, E-commerce, Hospital
 - Frequent writes. Normalization preferred.
- Denormalization is good for OLAP (Online Analytical Processing)
 - Reports, Dashboards, Business Intelligence, Analytics
 - Reads dominate. Denormalization preferred.
- **Denormalization** is the process of intentionally introducing redundancy into a database to improve read performance.
- Normalized
 - Read: Slow
 - Write: Fast
- Denormalized
 - Read: Fast
 - Write: Slow

Star Schema

Denormalized dimensions

Faster queries

More storage

Simple

Snowflake Schema

Normalized dimensions

Less redundancy

Less storage

Complex

- The **Entity-Relationship (ER) Model** is a conceptual data model used to represent the structure of a database using:
 - Entities
 - Attributes
 - Relationships
- Cardinality

- Defines how many instances of one entity can be associated with another.
- One-to-One
 - One person has one passport.
 - One passport belongs to one person.
 - There is only one entity on each side
- One-to-Many
 - One company has many employees.
 - Employee works for one company.
- Many-to-Many
 - One student takes many courses.
 - One course has many students.
 - In RDBMS, Many-to-Many is implemented using a junction table.

Participation Constraint

- Specifies whether participation is mandatory.
- Total Participation
 - Every employee must belong to a department. Mandatory.
- Partial Participation
 - Some employees may not have parking spaces. Optional.

TRANSACTIONS

- A **Transaction** is a sequence of one or more SQL operations that are executed as **one logical unit of work**.
- Either:
 - everything succeeds or everything fails
- Transaction Lifecycle
 - BEGIN
 - SQL Statement s
 - ...
 - COMMIT

or

 - BEGIN
 - SQL Statements
 - ...
 - ROLLBACK

- BEGIN Transaction: Starts a transaction.
 - BEGIN;
 - START TRANSACTION;
- COMMIT: Makes all changes permanent.
 - COMMIT;
- ROLLBACK: Cancels everything since transaction started.
 - ROLLBACK;
- SAVEPOINT:
 - Sometimes we don't want to rollback everything. Instead rollback to specific point.
 - SAVEPOINT {save_point_name}
 - ROLLBACK {save_point_name}
- Auto Commit: Many databases default to Auto Commit.
- To control transactions, disable auto-commit or explicitly start a transaction (behavior depends on the DBMS and client).
- Transaction States:
 - Active
 - Transaction currently executing.
 - Partially Committed
 - All SQL statements executed.
 - Waiting to make changes permanent.
 - Committed
 - Transaction successful. Changes saved permanently.
 - Terminated
 - Transaction finished.
 - Resources released.
 - Active
 - Failed
 - Transaction successful. Changes saved permanently.
 - Aborted
 - Rollback completed.
 - Database restored.
 - Terminated
- What Happens if Database Crashes?
 - Database checks transaction status. Committed? Keep changes. Not committed? Rollback. This is possible because of recovery mechanisms (such as transaction logs)

- **Transactions** ensure that a sequence of operations behaves as one logical unit.
- But what happens when **multiple transactions execute at the same time?**
 - Concurrency Control deals with it.

Concurrency Control

- **Concurrency Control** is the mechanism used by a DBMS to ensure that multiple concurrent transactions execute correctly without violating data consistency.
- Transaction Schedule
 - A **Schedule** is the order in which operations from multiple transactions execute.
- Types of Schedule
 - Serial: One after another. Safe, simple, slow.
 - Concurrent: Transaction overlap. Fast, needs concurrency control.
- Serializability
 - A concurrent schedule is **Serializable** if its result is the same as some serial execution of those transactions.
- Why Serializability?
 - Because serial execution is always correct. Concurrency Control tries to achieve serial-like correctness without sacrificing performance.

DEADLOCK

- What happens if two transactions are waiting for each other forever? This situation is called a **Deadlock**.
- A **Deadlock** occurs when two or more transactions are waiting indefinitely for each other to release resources.

Deadlock Conditions (aka Coffman Conditions), a deadlock can occur **only if all four of these conditions hold simultaneously**:

- Mutual Exclusion
 - A resource can be used by only one transaction at a time.
 - Without Mutual Exclusion, deadlocks cannot occur.
- Hold and Wait
 - Transaction already holds one resource while waiting for another.
- No Pre-emption
 - Resources cannot be forcibly taken away.
 - Only transaction holding lock can release it.

- Circular Wait
 - Transaction A waits for B.
 - Transaction B waits for C.
 - Transaction C waits for A.
 - Cycle.
- A **Wait-For Graph (WFG)** represents waiting relationships between transactions.
 - Nodes = Transactions
 - Edges = Waiting
- Graph contains cycle = Deadlock exists
- DBMS periodically checks Wait-For Graph. If cycle found = Deadlock detected.

Deadlock Recovery

- Most common approach: Choose one transaction as Victim. Rollback victim. Release locks. Other transaction continues.
- Choosing Victim:
 - DBMS tries to minimize cost.
 - Factors include:
 - Amount of work already done
 - Number of rows modified
 - Number of locks held
 - Estimated rollback cost
 - Transaction priority

Deadlock Prevention

- Instead of detecting, prevent deadlock. Break one Coffman Condition.
- Resource Ordering
 - Always acquire resources in same order.
 - $A \Rightarrow B$, but not in reverse order. No we can never have circular wait.
- Lock Timeout
 - Suppose transaction waits too long. Database automatically aborts it.

Deadlock Avoidance

- Different from prevention. Uses additional information to decide whether granting a lock is safe.
- Classic algorithm: Banker's Algorithm

-

- **Deadlock:** Transactions wait forever because they wait for each other.
- **Starvation:** Transaction never gets resource because others always get priority.
- **Functional Dependencies:**
 - Trivial
 - Non Trivial
 - Full
 - Partial
 - Transitive

QUERY PROCESSING

- **Query Processing** is the sequence of steps a DBMS performs to convert an SQL query into an efficient execution plan and produce the final result.

QUERY OPTIMIZATION

- **Query Optimization** is the process of finding the most efficient execution plan for a SQL query.
- **Goals of Query Optimization:**
 - CPU time
 - Disk I/O
 - Memory usage
 - Temporary storage
 - Network communication
 - Total execution time
- DBMS uses pages to store data physically instead of storing everything in a single large file

PARTITIONING

- **Partitioning** is the process of dividing a large table or index into smaller, independent pieces called **partitions**, while allowing applications to query the table as if it were a single table.
- Partitioning provides: (Benefits)
 - Faster queries (when only a few partitions are needed)
 - Easier maintenance
 - Faster backups/restores (at the partition level)
 - Easier archival

- Faster bulk deletes
- Better scalability
- Partitioning vs Sharding
 - Partitioning means splitting data inside one database.
 - Sharding means splitting data across multiple database servers.
- Partition Pruning:
 - Without Partitioning Search All Partitions
 - With Partition Pruning Search Only Relevant Partition

Type of Partitioning

- Range Partitioning
 - Most common. Rows assigned based on a range.
- List Partitioning
 - Rows assigned based on specific values.
 - PARTITION BY LIST(department): HR, Sales, etc.
- Hash Partitioning
 - Uses Hash Function.
 - Good for Evenly distributing data.
 - Poor for Range queries.
- Composite Partitioning
 - Uses multiple strategies.
 - First range, then hash
- Horizontal vs Vertical Partitioning
 - Horizontal Partitioning: Rows Splitting
 - Vertical Partitioning: Column Splitting
- Local Index: Each partition has it's own index
- Global Index: Single index covers all partitions.

REPLICATION

- **Replication** is the process of maintaining multiple copies of the same database on different servers.
- Replication provides: (Goals of Replication)
 - High Availability
 - Fault Tolerance
 - Load Balancing
 - Disaster Recovery

- Faster Reads
 - Geographic Distribution
- If Replication solved the problem of high availability and read scalability, then Sharding solves the problem of unlimited database growth.

Sharding

- **Sharding** is the process of splitting a database horizontally across multiple independent database servers called **shards**.
- Each shard stores **only a portion of the data**.
- Example, if users table contain 10 Million users instead of one server.
 - Shard 1: 1 to 3 Million Users
 - Shard 2: 4 to 6 Million Users
 - Shard 3: 7 to 10 Million Users
- Each server stores only its assigned users.
- Adding more data into a single server is said to be vertical scaling.
- Adding more servers to store more data is said to be horizontal scaling.
- Sharding enables horizontal scaling.
- Sharding provides: (goals of sharding)
 - Unlimited storage growth
 - Higher write throughput
 - Better scalability
 - Load distribution
 - Fault isolation
 - Geographic distribution (depending on design)
- Sharding vs Partitioning
 - Partitioning splits data inside one database.
 - Sharding splits data across multiple database servers.
- A **Shard Key** is the attribute used to determine which shard stores a particular row.
- Types of Sharding:
 - Range Sharding
 - Hash Sharding
 - Directory Sharding
 - Geo Sharding
- **Recovery** is the process of restoring a database to a correct and consistent state after a failure.

- CIA Triad
 - Confidentiality
 - Only authorized users can access data.
 - Integrity
 - Data must remain correct and unchanged unless modified by authorized operations.
 - Availability
 - Authorized users must be able to access the database when needed.
- GRANT Command: GRANT Command.
- REVOKE Command: Removes permissions.
- Preventing SQL Injection:
 - Never concatenate user input into SQL strings. Use Parameterized Queries / Prepared Statements.

TRIGGERS

- A **Trigger** is a stored database object that automatically executes in response to a specified database event.
- When an event happens, we can setup a trigger, for example insert, update, delete to store or log the data before and after the event
- We can create a trigger or let the database do it automatically
- Types of Triggers
 - INSERT Trigger
 - UPDATE Trigger
 - DELETE Trigger
- Timing of Trigger
 - BEFORE
 - Runs before the SQL statement is applied.
 - AFTER
 - Runs after the SQL statement has successfully completed.
 - INSTEAD OF
 - Mainly used on Views. Instead of executing the original statement, trigger executes custom logic.

```
CREATE TRIGGER trigger_name
[BEFORE | AFTER] [INSERT | UPDATE | DELETE]
ON table_name
FOR EACH ROW
BEGIN
    -- Your executable trigger code goes here
END;
```

```
CREATE TRIGGER trigger_name
AFTER INSERT
ON employees
FOR EACH ROW
BEGIN
    -- Trigger Logic
END;
```

```
CREATE TRIGGER log_user_signup
AFTER INSERT ON users
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (user_id, action, action_time)
    VALUES (NEW.id, 'SIGNUP', CURRENT_TIMESTAMP);
END;
```


- Row-Level Trigger
 - Runs once for every affected row.
- Statement-Level Trigger
 - Runs once per SQL statement, regardless of how many rows are affected.
- Recursive Triggers
 - Suppose Trigger updates same table. That update fires trigger again.
Possible infinite loop.
- Trigger Order
 - Multiple triggers can exist for same event. Some DBMSs allow controlling execution order. Others have predefined behavior.

Stored Procedure

- A **Stored Procedure** is a named collection of SQL statements stored inside the database that can be executed whenever needed.

General Syntax

```
</> SQL

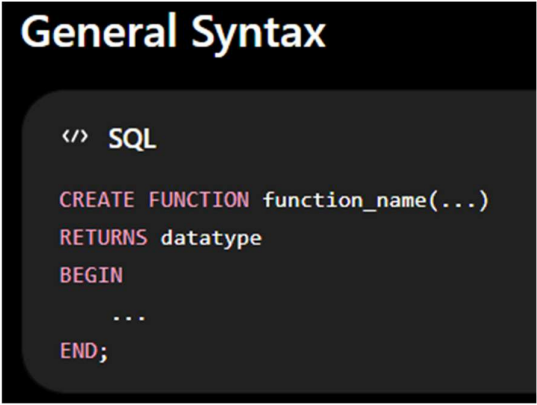
CREATE PROCEDURE procedure_name(...)
BEGIN
    SQL Statements;
END;
```

- Calling a procedure:

```
CALL TransferMoney(101,102,5000);
```

FUNCTIONS

- A **Function** is a stored database object that performs a computation and **returns a value**.



The image shows a dark-themed code editor with the title 'General Syntax' in yellow. Below the title, there is a code block for creating a function in SQL. The code is as follows:

```
</> SQL

CREATE FUNCTION function_name(...)
RETURNS datatype
BEGIN
    ...
END;
```

- Calling a function:
`SELECT calculate_bonus(50000);`
- Some DBMSs allow procedures to return values through **OUT** parameters.

CURSOR

- A **Cursor** is a database object that allows a program or stored procedure to retrieve and process query results **one row at a time**.
- Result Set
 - A cursor operates on a Result Set.
 - Example: `SELECT name, salary FROM employees;`
 - Result Set: (Alice, Bob, Charlie)
 - Cursor moves through these rows.
- Cursor Life Cycle: every cursor has four steps.
 - Declare
 - Tell database which query cursor will execute.
 - Open
 - Execute the query. Database creates result set.
 - Fetch
 - Retrieve one row. And then another so on.
 - Close
 - Release memory and resources.
- Uses a current pointer while moving to and fro in the result set

Types of Cursors

- Forward Only Can move only forward.
 - Scrollable Can move forward and backward.
 - Read Only Can only read data. Cannot update rows.
 - Updateable Allows reading and modifying current row.
-
- Implicit Cursor We don't create them, DB create internal cursors.
 - Explicit Cursor Created manually.

NoSQL

- SQL Fixed Schema
- NoSQL Flexible Schema

Types of NoSQL Databases

- Key-Value
 - Document
 - Column Family
 - Graph
-
- SQL One Big Server (Vertical Scaling)
 - NoSQL Designed to scale horizontally (Server1, Server2, Server3, etc.)

ACID	BASE
Strong consistency	Basically Available
Immediate consistency	Soft State
Strict transactions	Eventually Consistent (in many systems)

ACID

- Atomicity
- Consistency
- Isolation
- Durability